

SiteController.cs

Adım adım

Ne işe yarıyor?

Neden böyle yazıldı?

Hangi veri, hangi ekrana gidiyor?

Ders 01 · Başlangıç seviyesi

Gerçek proje kodu, örnek URL'ler ve cevaplı alıştırmalar.

Voku CMS · 24 Eylül 2026

Kod sürümü: 4c8cf8c

Kapsam: Yalnızca SiteController.cs

İçindekiler

01 Önce büyük resmi görelim

02 using, namespace ve sınıf

03 SiteDbContext db nasıl geliyor?

04 URL hangi metoda gidiyor?

05 Render neden ortak bir metot?

06 İlk kayıt sorgusunu parçalayalım

07 Kayıt bulunamazsa ne olur?

08 Hangi ekran türündeyiz?

09 query ve kategori listesi

10 Kategori filtresi nasıl uygulanıyor?

11 Kaç kayıt, kaç sayfa?

Başlıklara tıklayarak ilgili bölüme geçebilirsiniz. İlk okumada bölümleri sırayla takip edin; son iki bölüm dosyanın tam kodunu içerir.

İçindekiler · devam

- 12 Sıralama, Skip ve Take
- 13 View'a tam olarak ne gönderiliyor?
- 14 Blog detayında header nereden geliyor?
- 15 Diğer sayfalar ve tam istek örnekleri
- 16 Kendi başınıza deneyin
- 17 SitePageViewModel neden ayrı?
- 18 Page.cshtml: modelden ekran seçmek
- 19 _BlogListing: kart, filtre ve sayfalama
- 20 Bir alanın admin'den ziyaretçiye yolculuğu
- 21 Kodun tamamı · ilk bölüm
- 22 Kodun tamamı · ikinci bölüm

Başlıklara tıklayarak ilgili bölüme geçebilirsiniz. İlk okumada bölümleri sırayla takip edin; son iki bölüm dosyanın tam kodunu içerir.

Önce büyük resmi görelim

Bu dosyanın görevi ne, neyi yapmıyor?

Bir ziyaretçi /about, /blog veya /blog/ilk-yazim adresini açtığı anda, hangi içeriğin bulunacağını ve ekrana hangi verilerin gönderileceğini **SiteController** belirler. Bu dosya ziyaretçinin okuma tarafıdır; güncellenen ders view ve model bağlantılarını da izler. Admin formundan kaydetme işlemi burada yapılmaz.

```
Tarayıcı URL ister
↓
Route uygun action'ı seçer
↓
SiteController veritabanından gerekli kayıtları ister
↓
SitePageViewModel ekran verilerini bir araya getirir
↓
Views/Site/Page.cshtml HTML üretir
↓
Tarayıcı bu HTML'yi gösterir
```

Bu PDF yalnızca Voku.Web/Controllers/SiteController.cs dosyasını öğretir. Başka dosyalara, bu dosyanın bir satırını anlayabilmek için gerektiği kadar bakacağız. Sonraki dersin konusunu siz belirleyebilirsiniz. Anlatım **4c8cf8c** sürümündeki mevcut koda dayanır.

İsim	Aklınızda nasıl tutabilirsiniz?
Controller	İsteği karşılayan ve yapılacak işi seçen sınıf.
Action	Bir URL/HTTP yöntemiyle dışarıdan çağrılan controller metodu.
Entity	Veritabanındaki kaydın C# nesnesi; burada ContentPage.
View model	Ekrana birlikte gönderilecek verilerin paketi; SitePageViewModel.
View	Bu paketi kullanıp HTML oluşturan Razor dosyası.

Controller HTML sayfasının kendisi değildir. “Şu view’ı, şu verilerle oluştur” kararını verir. Veritabanına erişebilmesi de bütün kayıtları her seferinde getirdiği anlamına gelmez.

Nasıl çalışın? PDF ile SiteController.cs dosyasını yan yana açın. Her bölümde örnek URL’yi takip edin. Önce satırın sonucunu kendi cümlemlerle söyleyin, sonra açıklamayla karşılaştırın. Kod kutularındaki satır numaraları öğretim içindir; kopyalarken numaraları dahil etmeyin.

using, namespace ve sınıf

Dosyanın ilk sekiz satırını okuyalım.

```
01 using Microsoft.AspNetCore.Mvc;
02 using Microsoft.EntityFrameworkCore;
03 using Voku.Web.Data;
04 using Voku.Web.Models;
05 namespace Voku.Web.Controllers;
06 public class SiteController(SiteDbContext db) : Controller
07 {
08     private const int PageSize = 6;
```

Kod	Anlamı / neden var?
using Microsoft.AspNetCore.Mvc	Controller, IActionResult ve HttpGet gibi MVC adlarını kısa yazabilmek için.
using Microsoft.EntityFrameworkCore	AsNoTracking, ToListAsync gibi EF uzantı metotlarına erişmek için.
using Voku.Web.Data	SiteDbContext ve BlogMarkup bu namespace altında.
using Voku.Web.Models	SitePageViewModel gibi modelleri kullanmak için.
namespace Voku.Web.Controllers;	Sınıfın kod içindeki ad alanı. Tek başına URL tanımlamaz.
public class SiteController	Dışarıdan erişilebilen SiteController adlı sınıfın tanımı.
: Controller	MVC Controller sınıfından miras alır; View, NotFound, Problem gibi metotları kullanır.
private const int PageSize = 6	Bu sınıfa özel, değişmeyen tam sayı: bir listede en fazla 6 yazı.

using burada kod adlarını erişilebilir hale getirir; veritabanı sorgusu başlatmaz. Paket yüklemekle aynı işlem değildir. Paket bağımlılıkları csproj içinde tanımlanır.

class bir sınıf tanımlar. Süslü parantezler sınıfın veya metodun sınırını gösterir. Noktalı virgül bir ifadenin sonunu belirtir. const değerinin uygulama çalışırken değişmesini önler. PageSize'ı 9 yapıp yeniden derlerseniz hem ana sayfa limiti hem blog sayfa büyüklüğü değişir.

Bu bölümün sorusu: Controller dosyasına using ekledim; veritabanı okunmuş olur mu? Hayır. Sadece ilgili tür/metot isimlerini kullanabilirim.

SiteDbContext db nasıl geliyor?

db veritabanının kendisi değil, erişim nesnesidir.

```
public class SiteController(SiteDbContext db) : Controller
```

Parantez içi bu sınıfın **primary constructor** tanımıdır. “SiteController oluşturulurken bana SiteDbContext türünde bir nesne ver” der. Nesneye sınıf içinde db adıyla erişiriz. db özel bir C# kelimesi değildir; context adını da verebilirdik.

Bu nesneyi ASP.NET Core’un dependency injection sistemi sağlar. Program.cs’de AddDbContext<SiteDbContext>(...UseSqlite...) kaydı, context’in nasıl oluşturulacağını tarif eder. Controller içinde bağlantı yolunu tekrar yazmamız gerekmez.

```
// Aynı fikrin klasik constructor ile gösterimi:
public class SiteController : Controller
{
    private readonly SiteDbContext db;

    public SiteController(SiteDbContext db)
    {
        this.db = db;
    }
}
```

Bu örnekte this.db sınıf alanıdır; sağdaki db constructor parametresidir. Mevcut dosya daha kısa primary constructor yazımını kullanıyor. İki örneği aynı dosyaya birlikte eklemeyin; bu yalnızca anlamı göstermek için alternatif bir yazımdır.

İfade	Ne anlatır?
db	İstek sırasında kullanılan SiteDbContext nesnesi.
db.Pages	ContentPage kayıtları için EF sorgu/işlem giriş noktası.
SQLite dosyası	Gerçek kalıcı veri; varsayılan olarak App_Data/voku.db.
db.Pages.Where(...)	Hangi kayıtların isteneceğini tarif eden sorgu.

Bu satır veritabanına erişimi mümkün kılar; kendi başına SQL çalıştırmaz. Gerçek veri okuma SingleOrDefaultAsync, CountAsync ve ToListAsync gibi sorguyu yürüten işlemlerde olur.

URL hangi metoda gidiyor?

Action, route ve parametre kavramlarını üç URL ile öğrenelim.

```
09 [HttpGet("/")]
10 public Task<IActionResult> Home() => Render("home", false, null, 1);
11 [HttpGet("/{slug}")]
12 public Task<IActionResult> Page(string slug, [FromQuery] string? category, [FromQuery(Name =
"page")] int number = 1)
13     => Render(slug, false, category, number);
14 [HttpGet("/blog/{slug}")]
15 public Task<IActionResult> Detail(string slug) => Render(slug, true, null, 1);
16 [Route("/server-error")]
17 public IActionResult ServerError() => Problem("İşlem tamamlanamadı. Lütfen tekrar deneyin.");
```

İstek	Çalışan action	Render'a gönderilen değerler
GET /	Home()	"home", false, null, 1
GET /about	Page(...)	"about", false, null, 1
GET /blog?category=Gezi&page=2	Page(...)	"blog", false, "Gezi", 2
GET /blog/ilk-yazim	Detail(...)	"ilk-yazim", true, null, 1

[HttpGet("/{slug}")] içindeki süslü parantez URL'den alınacak değişkeni belirtir. /about için slug=about, /blog için slug=blog olur. /blog/ilk-yazim iki parçalı olduğundan /blog/{slug} route'u ile Detail action'ına gider.

[FromQuery], değer URL'de ? işaretinden sonraki sorgu parametrelerinden alınacağını belirtir. category=Gezi → category değişkeni. [FromQuery(Name="page")] int number=1 ise URL'de adı page olan değeri C# tarafında number değişkenine bağlar. Parametre verilmezse başlangıç değeri 1 olur.

string? ifadesi null olabilen string demektir; ziyaretçi kategori seçmemiş olabilir. => burada kısa metod gövdesidir: metod sağdaki ifadenin sonucunu döndürür. "Parametreleri hazırla ve Render'a aktar" işini kısa yazar.

ServerError, Problem(...) ile hata yanıtı üretir; Page.cshtml'i açmaz. Program.cs'de Development dışındaki exception handler bu adrese yönlendirilmiştir. Normal içerik route'larından farklı bir hata işleme yoludur.

Buradaki public action'lar giriş kapılarıdır. Render private bir yardımcı metottur; /Render diye bir endpoint oluşturulmaz.

Render neden ortak bir metot?

Aynı işlemleri üç farklı action'da tekrar yazmamak için.

```
private async Task<IActionResult> Render(
    string slug, bool detail, string? category, int number)
```

Parça	Bu metottaki anlamı
private	Yalnızca bu sınıf içinden çağrılan yardımcı metot.
async	Metot içinde await kullanılabilmesini sağlar.
Task<IActionResult>	Tamamlandığında bir MVC sonucu üreten asenkron işlem.
IActionResult	View, NotFound veya benzeri HTTP yanıtı sonucunun ortak türü.
slug	Bulunacak içeriğin adres parçası.
detail	false: sabit sayfa; true: blog detayı.
category	Blog listesine uygulanabilecek kategori filtresi.
number	İstenen liste sayfasının numarası.

Home, Page ve Detail farklı adreslerden aynı işe başlar: içeriği bul, yayın durumunu kontrol et, ekrana uygun verileri hazırla. Ortak kısmın Render içinde olması aynı sorgu/hata kodunun üç kez yazılmasını önler.

```
// Action işi doğrudan döndürüyor:
public Task<IActionResult> Home()
    => Render("home", false, null, 1);

// Render içinde sorgunun sonucu kullanılacağı için await var:
var count = await query.CountAsync();
```

Home, Task'ı doğrudan döndürdüğü için kendi içinde await kullanmıyor. Bu, işin unutulup arka planda bırakıldığı anlamına gelmez; ASP.NET Core dönen işlemin tamamlanmasını bekler. Render ise count gibi sonuçları sonraki satırlarda kullandığı için await eder.

await, sonucu hazır olduğunda metodun devam etmesini sağlar. Sorguları otomatik paralelleştirmez ve SQL'in mutlaka daha hızlı çalışmasını sağlamaz. Burada önemli olan, sonuç isteyen satırların doğru sırada tamamlanmasıdır.

Akılda tutun: Controller'daki Render adı geliştiricinin seçtiği bir isimdir. HTML'yi doğrudan bu metot çizmez; sonunda dönen View sonucu Razor ekranını oluşturur.

İlk kayıt sorgusunu parçalayalım

Tek sayfa kaydı nasıl seçiliyor?

```
var page = await db.Pages.AsNoTracking()
    .SingleOrDefaultAsync(p =>
        p.Slug == slug &&
        p.IsDetail == detail &&
        p.Published);
```

1. db.Pages: Pages tablosundaki ContentPage kayıtlarını sorgula.
2. AsNoTracking(): Bu nesneyi EF değişiklik takibine alma. Bu controller yalnızca okuyup gösteriyor.
3. p => ...: Her kaydın sağlaması gereken koşulu tarif eden lambda. p adı pageRecord da olabilirdi.
4. p.Slug == slug: Kaydın slug alanı, metodun slug parametresine eşit olsun.
5. p.IsDetail == detail: Kayıt türü gelen detail parametresine eşit olsun; her zaman true demiyoruz.
6. p.Published: bool özellik doğrudan koşulda kullanılmış; p.Published == true anlamına gelir.
7. SingleOrDefaultAsync: Tek eşleşme varsa kaydı, yoksa null getir. Birden fazla eşleşme varsa hata ver.
8. await: Sorgu sonucunu alıp page değişkenine ata.

Örnek çağrı	Aranan kayıt
Render("about", false, null, 1)	Slug=about, IsDetail=false, Published=true
Render("ilk-yazim", true, null, 1)	Slug=ilk-yazim, IsDetail=true, Published=true

var “türsüz değişken” demek değildir; C# türü sağdaki ifadedden çıkarır. Buradaki page tek bir ContentPage nesnesi veya null olabilir. List<ContentPage> değildir. && bütün koşulların birlikte doğru olmasını ister.

AsNoTracking veritabanını salt okunur yapmaz; context’in bu sorgudan gelen nesneyi takip etmemesini sağlar. Okuma ekranı için uygundur. (IsDetail, Slug) birleşik benzersiz indeksi normal koşullarda aynı türde birden çok slug kaydı oluşmasını önler.

Kavramsal SQL: SELECT ... FROM Pages WHERE Slug=@slug AND IsDetail=@detail AND Published=1. Bu öğretici bir özet; EF’nin ürettiği SQL’in birebir kopyası değildir.

Kayıt bulunamazsa ne olur?

null kontrolü, 404 ve erken return.

```
22         if (page == null)
23         {
24             Response.StatusCode = 404;
25             page = await db.Pages.AsNoTracking().SingleOrDefaultAsync(p => p.Slug == "error" &&
                !p.IsDetail && p.Published);
26             if (page == null) return NotFound();
27         }
```

Önceki sorguda kayıt yoksa page=null olur. Bu hem gerçekten olmayan bir slug için hem de mevcut ama Published=false olan bir yazı için geçerlidir. Ziyaretçi tarafında taslak yazı da bulunamamış gibi davranır.

1. Response.StatusCode=404: HTTP yanıtının bulunamadı durumunu ayarla.
2. İkinci sorguda Slug=error, IsDetail=false, Published=true kaydını ara.
3. Varsa page değişkenine bu hata sayfasını koy ve aşağıdaki normal view akışına devam et.
4. Bu kayıt da yoksa return NotFound(): metottan hemen çık ve 404 sonucu ver.

page değişkeninin değeri değişebilir: önce aranan sayfa null'dı; sonra hata sayfası kaydı atanabilir. Ama Response.StatusCode 404 kalır. Böylece kullanıcı tasarlanmış hata ekranını görürken istemciye doğru HTTP durum kodu gider.

İstek / durum	Sonuç
/olmayan-sayfa, error kaydı yayında	Error içeriği + HTTP 404
/olmayan-sayfa, error kaydı da yok	NotFound sonucu + HTTP 404
/blog/taslak-yazi	Ziyaretçi açısından bulunamadı; 404
/error doğrudan açılırsa ve kayıt yayındaysa	Normal sayfa kaydı bulunur; genellikle HTTP 200

return, yalnızca bir değer söylemez; o metodun çalışmasını o noktada bitirir. Bu nedenle NotFound döndükten sonra aşağıdaki listeleme kodu çalışmaz.

Mini kontrol: Kayıt bulunamadığında neden hemen her zaman NotFound dönmüyoruz? Çünkü veritabanından yönetilebilen error sayfasının tasarımını göstermek istiyoruz. Ama hata sayfası bulunmasa bile 404 üretebilen bir yedek yolumuz var.

Hangi ekran türündeyiz?

Ana sayfa, blog listesi, detay ve diğer sayfalar ayrılıyor.

```
if (!page.IsDetail && page.Slug is "home" or "blog")
{
    // Listeleme verileri hazırlanır ve view döndürülür.
}
if (page.IsDetail)
{
    // Detay sayfası verileri hazırlanır ve view döndürülür.
}
return View("Page", new SitePageViewModel { Page = page });
```

!page.IsDetail ifadesi IsDetail false olsun demektir. Başındaki ünlem mantıksal “değil” operatörüdür. page.Slug is "home" or "blog" C# örüntü eşleştirmesiyle slug bu iki değerden biri mi diye sorar.

```
// Aynı koşulun daha açık yazımı:
if (!page.IsDetail &&
    (page.Slug == "home" || page.Slug == "blog"))
```

Kayıt	Seçilen dal
home + IsDetail=false	Son yazıların listesi
blog + IsDetail=false	Kategori ve sayfalama içeren blog listesi
ilk-yazim + IsDetail=true	Blog detay ekranı
about + IsDetail=false	En alttaki genel sayfa sonucu
error + IsDetail=false	En alttaki genel sayfa sonucu; hata akışıdaysa 404 korunur

Kod if / else if yerine iki if ve sondaki return ile yazılmış. İlk dalın içinde return olduğu için o dal çalışınca ikinci if'e geçilmez. Detay dalı çalışınca da alttaki genel return'e ulaşılmaz. Bu “erken dönüş” yaklaşımı fazla iç içe kod yazmayı azaltır.

Üç dal da aynı Page adlı Razor view'ını kullanır; fakat view modelin doldurulan alanları farklıdır. Liste ekranında Posts ve Categories gerekir, detayda HeaderHtml/FooterHtml gerekir, normal sayfada çoğunlukla Page yeterlidir.

Karıştırmayın: page.Slug == "blog" bir blog yazısı değil, blog yazılarını listeleyen sabit sayfadır. Gerçek blog yazılarının IsDetail değeri true olur.

query ve kategori listesi

Where, Select, Distinct, OrderBy ve ToListAsync birlikte nasıl çalışıyor?

```
30         var query = db.Pages.AsNoTracking().Where(p => p.IsDetail && p.Published);
31         var categories = await query.Where(p => p.Category != null && p.Category != "")
32             .Select(p => p.Category!).Distinct().OrderBy(c => c).ToListAsync();
```

İlk satır query adlı temel sorguyu kurar: yalnızca yayınlanmış blog detayları. Bu noktada henüz sonuç listesi alınmaz. Sonraki satırlar bu temel sorgudan kategori isimlerini çıkaran ayrı bir sorgu oluşturur.

Yazı başlığı	Category	Published
İstanbul gezisi	Gezi	true
EF Core başlangıcı	Yazılım	true
Ankara gezisi	Gezi	true
İsimsiz kategori	null	true
Taslak haber	Haber	false

```
query
// Yayınlanmış dört yazıyı hedefler; taslak haber yok.
.Where(p => p.Category != null && p.Category != "")
// Kategorisiz kaydı eler.
.Select(p => p.Category!)
// Artık yazı nesneleri değil: "Gezi", "Yazılım", "Gezi"
.Distinct()
// Tekrarsız: "Gezi", "Yazılım"
.OrderBy(c => c)
// Kategori değerine göre artan sıra.
.ToListAsync()
// Sorguyu çalıştır; await ile List<string> al.
```

Select hangi alanı alacağını seçer; kayıt silmez. Distinct sonuçtaki tekrarları kaldırır; veritabanındaki aynı kategorili yazılar aynen kalır. OrderBy(c => c) içindeki c artık ContentPage değil string kategori adıdır. Sıralamanın ayrıntısı veritabanının metin karşılaştırma kurallarına bağlıdır.

Category! sonundaki ünlem, derleyiciye “burada null olmadığını kabul et” der; çalışma anında null kontrolü yapmaz. Önceki Where null değerleri elemiştir. Bu ünlem, önceki bölümdeki !page.IsDetail ile aynı görevde değildir.

Neden kategoriler seçili filtre uygulanmadan önce alınıyor? Gezi seçtiğinizde filtre menüsünde Yazılım da kalabilirdi. Aksi halde kullanıcı sadece seçili kategoriyi görebilirdi.

Kategori filtresi nasıl uygulanıyor?

Kategorileri listelemek ile yazıları kategoriye göre filtrelemek farklı işlerdir.

```
33         category = page.Slug == "blog" ? category?.Trim() : null;
34         if (!string.IsNullOrEmpty(category)) query = query.Where(p => p.Category == category);
```

İlk satır üçlü koşul operatörü kullanır. Yapısı koşul ? doğruysa : yanlışsa biçimindedir. Blog sabit sayfasındaysak gelen kategori değerini temizler; ana sayfadaysak category=null yapar. Böylece ana sayfa her zaman genel son yazıları gösterir.

```
// category = " Gezi " ise:
category?.Trim() // "Gezi"

// category = null ise:
category?.Trim() // null; Trim çağrılmaz.
```

?. null koşullu erişimdir. Değer varsa metodu çağırır, yoksa null verir. Trim baştaki/sondaki boşlukları kaldırır; kategori adının içindeki boşlukları kaldırmaz. string.IsNullOrEmpty, null veya boş string mi diye kontrol eder. Önündeki ! nedeniyle “boş değilse filtre uygula” diyoruz.

query = query.Where(...), temel sorguya ek şart koyar. Böylece hem IsDetail/Published koşulları hem Category koşulu geçerli olur. Bu bir veritabanı UPDATE işlemi değildir; sadece çalıştırılacak SELECT sorgusunun kapsamı daralır.

URL	Yazı sorgusunda kategori koşulu
/blog	Yok; tüm yayınlanmış detaylar
/blog?category=Gezi	Category == "Gezi"
/blog?category=	Yok; temizlenmiş değer boş
/blog?category=Olmayan	Eşleşme yok; boş liste
/?category=Gezi	Home action bu filtreyi kullanmaz

categories çoğul: filtre menüsünün seçenekleri. category tekil: kullanıcının seçtiği değer. query: seçime göre yazıları getirecek sorgu. Bu üç değişken aynı şey değildir.

Düşünme sorusu: Bir yazıyı taslağa çekersek tek başına onun kullandığı kategori menüde kalır mı? Hayır; categories sorgusu da yalnızca Published=true yazılar üzerinden kuruluyor.

Kaç kayıt, kaç sayfa?

CountAsync, Ceiling, Max ve Clamp için sayısal örnekler.

```
35         var count = await query.CountAsync();
36         var totalPages = (int)Math.Ceiling(count / (double)PageSize);
37         number = Math.Clamp(number, 1, Math.Max(1, totalPages));
```

CountAsync() filtreye uyan toplam yazı sayısını getirir. Altı kayıtlık görünen sayfanın sayısı değildir. Örneğin seçili kategoride 14 yayınlanmış yazı varsa count=14 olur.

```
PageSize = 6
count = 14

14 / (double)6 = 2.333...
Math.Ceiling(2.333...) = 3
(int)3 = 3

totalPages = 3
```

(double), bölmeyi ondalıklı yapar. İki int doğrudan bölünseydi kesir atılır, 14/6 sonucu 2 olurdu. Ceiling yukarı yuvarlar; son iki yazı için üçüncü sayfanın gerektiğini hesaplar. (int) sonucu tam sayıya çevirir.

count	totalPages	Sayfalardaki kayıt sayıları
0	0	Hiç kayıt yok
1	1	1
6	1	6
7	2	6 + 1
14	3	6 + 6 + 2

Math.Clamp(number, 1, Math.Max(1, totalPages)) istenen sayfayı geçerli aralığa çeker. Clamp, alt sınırın altındaki değeri alt sınıra; üst sınırın üstündekini üst sınıra getirir. Max(1,totalPages), hiç kayıt yokken bile üst sınırın en az 1 olmasını sağlar.

İstek	totalPages=3 iken number
page=2	2
page=0 veya page=-8	1
page=99	3

Hiç yazı yoksa TotalPages=0 ama PageNumber=1 olur. Çelişki değil: veri için sıfır sayfa var; Skip hesabına geçerli bir başlangıç değeri veriyoruz. View bu durumda boş liste mesajı gösterir.

Sıralama, Skip ve Take

Tüm yazıları belleğe almadan istenen sayfayı getiriyoruz.

```
38         var posts = await query.OrderByDescending(p => p.PublishedAtUtc).ThenByDescending(p =>
39         p.Id)
        .Skip((number - 1) * PageSize).Take(PageSize).ToListAsync();
```

1. OrderByDescending(PublishedAtUtc): Tarihi yeni olan yazı önce gelsin.
2. ThenByDescending(Id): Tarihler aynıysa büyük Id önce gelsin.
3. Skip((number - 1) * PageSize): Önceki sayfalardaki kayıtları atla.
4. Take(PageSize): En fazla 6 kayıt al.
5. ToListAsync + await: Sorguyu çalıştır ve sonuçları posts listesine ata.

number	Skip hesabı	Alınan sıralı kayıtlar
1	$(1-1) \times 6 = 0$	1–6
2	$(2-1) \times 6 = 6$	7–12
3	$(3-1) \times 6 = 12$	13–18; yalnızca 14 varsa 13–14

ThenByDescending ikinci bir OrderBy yerine kullanılır. Amaç ilk tarih sırasını koruyup eşit tarihler içinde Id ile sıralamaktır. İkinci bir OrderBy yazmak ilk sıralamayı değiştirebilir. Id ile eşitliği çözmek sayfa sınırlarındaki sonucu daha belirli hale getirir.

Filtreleme, sıralama ve Skip/Take, ToListAsync öncesinde EF sorgusunun parçalarıdır; amaç veritabanından yalnızca gereken alt kümeyi almaktır. Önce ToListAsync yapıp ardından Where/Skip uygulaysaydık artık bellekteki listeyi işliyor olurduk ve gereksiz kayıtlar getirebilirdik.

Sıralama alanı PublishedAtUtc'dir; UpdatedUtc değildir. Yazının gövdesini düzenlemek onu otomatik en üste taşımaz. Yazı tarihini değiştirmek ise sıralamayı etkiler. Gelecekteki tarih için ayrıca bir "tarih geldi mi?" koşulu olmadığı için Published=true olan gelecek tarihli yazı da görünür.

ToListAsync sonucu List<ContentPage> olur. Ancak kategori sorgusunda Select ile string'e dönüştürdüğümüz için aynı ToListAsync orada List<string> üretmişti. Sonucun türünü sorgunun son eleman türü belirler.

View'a tam olarak ne gönderiliyor?

View adı ile view modelin alanlarını birbirinden ayıralım.

```
40         return View("Page", new SitePageViewModel { Page = page, Posts = posts, Categories =
categories,
41             Category = category, PageNumber = number, TotalPages = totalPages, TotalPosts = count
});
```

View("Page", ...) ifadesindeki "Page" bir view adıdır. SiteController için ilgili dosya **Views/Site/Page.cshtml** olur. İkinci parametre o view'a gönderilen veri nesnesidir.

new SitePageViewModel { ... } yeni bir ekran modeli oluşturur. Süslü parantez içindeki atamalar bu nesnenin alanlarını doldurur. Page = page ifadesinde soldaki Page model özelliği, sağdaki page ise veritabanından aldığımız yerel değişkendir.

View model alanı	Gönderilen değer	Ekrandaki amacı
Page	home/blog sabit sayfa kaydı	Başlık, açıklama ve sayfa HTML düzeni
Posts	O sayfaya ait en fazla 6 detay	Yazı kartları
Categories	Tekrarsız kategori isimleri	Kategori filtre bağlantıları
Category	Seçili kategori veya null	Seçili filtre ve sayfalama linkleri
PageNumber	Düzeltilmiş sayfa numarası	"Sayfa 2 / 3" bilgisi
TotalPages	Toplam sayfa sayısı	Önceki/sonraki kontrolleri
TotalPosts	Filtreye uyan toplam kayıt	"14 yazı" bilgisi

```
// Razor tarafında kavramsal kullanım:
@model SitePageViewModel

@foreach (var post in Model.Posts)
{
    <a href="@post.Url">@post.Title</a>
}
```

Mevcut Page.cshtml, listeleme ekranında bu modeli _BlogListing.cshtml partial view'ına aktarır. Kartların foreach döngüsü oradadır. Controller bu kartların HTML'sini tek tek üretmez; onları oluşturacak veri listesini sağlar.

Bu üç "page" farklıdır: URL'de page=2 sayfalama parametresidir; C# page değişkeni ContentPage kayıdır; View("Page") ise Razor dosyasının adıdır. Büyük/küçük harf ve bulunduğu bağlam anlamı değiştirir.

Blog detayında header nereden geliyor?

About Us örneğini gerçek kodla takip edelim.

```
43         if (page.IsDetail)
44         {
45             var home = await db.Pages.AsNoTracking().SingleOrDefaultAsync(p => p.Slug == "home" &&
!p.IsDetail);
46             return View("Page", new SitePageViewModel { Page = page,
47                 HeaderHtml = BlogMarkup.Chrome(home?.BodyHtml ?? page.BodyHtml, "header"),
48                 FooterHtml = BlogMarkup.Chrome(home?.BodyHtml ?? page.BodyHtml, "footer") });
49         }
```

IsDetail=true ise listeleme dalı atlanır ve bu blok çalışır. Detay yazısı ilk sorguda zaten page değişkenine alınmıştır. Burada ikinci kez ana sayfa kaydı bulunur; amaç detayın header ve footer alanlarını ana sayfaya paylaşmaktır.

```
home?.BodyHtml ?? page.BodyHtml
```

1. home varsa home.BodyHtml değerini al.
2. home null ise ?. nedeniyle sonuç null olur.
3. ??, sol taraf null ise sağdaki page.BodyHtml değerini kullanır.
4. BlogMarkup.Chrome(..., "header") seçilen HTML içinden header bölümünü çıkarır.
5. Aynı işlem footer için yapılır; iki string view modeline konur.

Chrome burada Google Chrome tarayıcısı değildir. Projede yazılmış yardımcı metodun adıdır; sayfanın üst/alt kabuk HTML'sini bulur. Bir web isteği atmaz ve yeni veritabanı sorgusu yapmaz. Kendisine verilen metin üzerinde çalışır.

Ana sayfanın BodyHtml alanında 2About Us varsa detay header'ında da aynı metin görünür. Çünkü veri doğrudan veritabanındaki ana sayfa HTML'sinden gelir. Admin'de ana sayfa menü metnini değiştirmek hem ana sayfaya hem blog detayına yansır.

Diğer sabit sayfalar bu dala girmediği için kendi BodyHtml içindeki header'ı kullanır. Bu yüzden onlar About Us gösterirken detay 2About Us gösterebilmişti. Bu bir CSS veya tarayıcı cache farkı olmak zorunda değildir; veri kaynağı farkıdır.

İki ince ayrıntı: Ana sayfa sorgusunda Published filtresi yok; burada ana sayfa chrome kaynağı olarak okunuyor. Ayrıca ?? yalnızca null durumunda fallback yapar. home mevcut ama HTML içinde header yoksa Chrome boş string döndürür; otomatik olarak detayın header'ına ikinci kez dönülmez.

Diğer sayfalar ve tam istek örnekleri

Aynı controller'ın üç farklı isteğe nasıl yanıt verdiğini tekrar izleyelim.

```
return View("Page", new SitePageViewModel { Page = page });
```

En alttaki satır Hakkımızda, Yetenekler, İletişim veya hata sayfası gibi genel içerikler içindir. Page alanı doldurulur. Posts ve Categories gibi alanlar SitePageViewModel'de başlangıçta boş listelerdir; bu sayfalar için blog listesi sorgusu çalıştırılmaz.

Adım	/about	/blog?category=Gezi&page=2	/blog/ilk-yazim
Action	Page	Page	Detail
İlk Render değerleri	about, false, null, 1	blog, false, Gezi, 2	ilk-yazim, true, null, 1
İlk kayıt	about sabit sayfası	blog sabit sayfası	ilk-yazim detayı
Ek sorgular	Yok	Kategoriler + count + posts	Ana sayfa header/footer kaynağı
View modeli	Page	Page + listeler + sayfalama	Page + HeaderHtml + FooterHtml
Görünüm	Genel BodyHtml	Dinamik yazı kartları	Dinamik detay başlığı ve yazı gövdesi

Başarılı normal /about isteğinde bir içerik sorgusu vardır. Başarılı listeleme isteğinde ilk sayfa sorgusu, kategori sorgusu, count sorgusu ve posts sorgusu olmak üzere dört içerik sorgusu yürütülür. Başarılı detay isteğinde detay kaydı ve ana sayfa kaydı için iki sorgu vardır. Bunlar bu controller'ın normal yollarıdır; başlangıç migration/seed işlemleri bu sayıya dahil değildir.

Kod yukarıdan aşağıya okunur ama her istek bütün satırları çalıştırmaz. Koşullar ve return ifadeleri izlenecek yolu belirler. Bu nedenle hata ayıklarken önce hangi URL'nin hangi parametrelerle geldiğini belirlemek, sonra o dalı izlemek önemlidir.

Controller'ın hiçbir dalında Add, Remove veya SaveChangesAsync yok. Bu dosya veriyi okur ve yanıt hazırlar. Admin'den değişen veriyi bir sonraki ziyaretçi isteğinde yeniden okuduğu için içerik güncel görünür.

Kendi başınıza deneyin

Küçük alıştırmalar; önce tahmin edin, sonra koddan kontrol edin.

Soru	Cevap ve gerekçe
detail=false ise IsDetail=true kayıt gelir mi?	Hayır. p.IsDetail == detail eşitlik koşulu vardır.
Published false kayda doğrudan URL ile gidilir mi?	Bu controller onu ilk sorguda dışlar ve 404 yoluna girer.
Select kaldırılırsa categories ne olur?	String listesi yerine entity sorgusu kalır; devamdaki OrderBy(c => c) kategori sıralaması olmaktan çıkar.
Distinct kaldırılırsa?	Aynı kategori her yazı için tekrarlanabilir.
13 yazı ve PageSize=6 ise kaç sayfa?	Ceiling(13/6.0)=3; 6+6+1 kayıt.
2. sayfa kaç kayıt atlar?	(2-1)×6=6.
Özet değişince neden kart güncellenir?	Controller güncel entity'yi Posts listesinde view'a yollar.
Başlık değiştiğinde bu dosyada SQL yazmalı mıyız?	Hayır. Okuma sorgusu güncel Title değerini zaten getirir.
Header niçin diğer sayfadan farklı olabilir?	Detay home.BodyHtml kaynağını, diğer sabit sayfa kendi BodyHtml kaynağını kullanır.
Query oluşturmak kayıtları değiştirmek midir?	Hayır. Sorgu tanıımıdır; burada veri yazma işlemi yok.

Breakpoint alıştırması: İlk kayıt sorgusuna breakpoint koyun. /about açın; slug=about ve detail=false değerlerini görün. Sonra /blog?category=Gezi&page=2 açıp category ve number değerlerini izleyin. Count sonrasında count ve totalPages'i, posts sorgusundan sonra listenin elemanlarını inceleyin.

Boş kategori alıştırması: /blog?category=OlmayanBirKategori açın. Ana blog sayfası kaydı yine bulunduğu için HTTP 404 beklemeyin. count=0 ve posts boş olur; view boş liste mesajı gösterir. "Sayfanın bulunamaması" ile "listenin boş olması" farklı durumlardır.

Kendi cümlemlerle tamamlayın: "Bu istek ... action'ına gider. İlk sorgu ... kaydını bulur. ... koşulu nedeniyle ... dalı çalışır. ... verileri SitePageViewModel'e konur. ... view'ı bu verileri HTML olarak gösterir."

Bir sonraki dosyaya geçmeden önce bu paragrafı /about, /blog ve /blog/ilk-yazim için üç kez doldurabilmeniz yeterlidir.

SitePageViewModel neden ayrı?

Tek entity, ekranın bütün ihtiyaçlarını karşılamıyor.

```
public required ContentPage Page { get; init; }
public IReadOnlyList<ContentPage> Posts { get; init; } = [];
public IReadOnlyList<string> Categories { get; init; } = [];
public string? Category { get; init; }
public int PageNumber { get; init; } = 1;
```

ContentPage tek bir kalıcı içerik kayıdır. Blog ekranında ise hem blog sabit sayfasının düzeni hem yazı listesi hem kategoriler hem de sayfalama bilgisi gerekir. SitePageViewModel, farklı kaynaklardan gelen bu verileri tek türü belli paket içinde taşır. Ayrı bir veritabanı tablosu değildir.

Yazım	Amaç / sınır
required Page	Nesne oluşturulurken Page atamasını derleyici düzeyinde ister; SQL NOT NULL veya MVC Required doğrulaması değildir.
get; init;	Nesne kurulurken atanır; sonradan normal property atamasını sınırlar. İçindeki entity'yi derinlemesine değişmez yapmaz.
IReadOnlyList<ContentPage>	View'a okuma odaklı liste arayüzü sunar. Altaki tüm nesneleri immutable hale getirmez.
= []	Boş liste başlangıcı; view'ın null kontrolü yerine Count/foreach kullanmasını kolaylaştırır.
string? Category	Filtre seçilmemişse null geçerli bir durumdur.

Posts listesi entity'ler içerir; kartlar Title, Description ve Url gibi mevcut alanları doğrudan okur. Ayrı BlogCardViewModel kullanmak gelecekte kartın ihtiyaç duyduğu alanları sınırlandırabilir; şu an böyle bir sınıf yoktur. Ekran modeli veritabanı modelinden ayrılmış olsa da içindeki her ögenin ayrı ekran modeline dönüştürülmesi zorunlu değildir.

Neden ViewBag yerine bu sınıf? Alan adlarını ve türlerini derleme sırasında kontrol edebilmek için. Model.Posts yanlış yazılırsa hata daha erken anlaşılır. Mevcut ziyaretçi controller'ı ViewBag.Posts kullanmıyor.

Page.cshtml: modelden ekran seçmek

Controller'ın gönderdiği paket Razor'da nasıl açılıyor?

```
@model SitePageViewModel
@{
    Layout = null;
    var entry = Model.Page;
    var isListing = !entry.IsDetail &&
        (entry.Slug == "home" || entry.Slug == "blog");
}
```

Model bütün ekran paketidir; entry bu paketin içindeki ContentPage kaydıdır. Model.Title yazmak yanlış olur çünkü Title SitePageViewModel'de değil, Model.Page üzerinde tanımlıdır. entry.Title bu uzun yolu kısaltır. Bu atama yeni veritabanı sorgusu yapmaz.

1. entry.IsDetail ise dinamik kapak, ana başlık, kategori/yazar/tarih ve makale gövdesi çizilir.
2. Ana sayfa veya blog listesi ise HTML içindeki BLOG_LIST işareti bulunur; öncesi çizilir, _BlogListing partial'ı eklenir, sonrası çizilir.
3. Diğer sabit sayfalarda kayıt gövdesi gösterilir.

```
@Html.Raw(Model.HeaderHtml)
<h1>@entry.Title</h1>
@Html.Raw(BlogMarkup.ArticleBody(entry.BodyHtml))

<partial name="_BlogListing" model="Model" />
```

Bu satırlar farklı dallardan alınmış örneklerdir. Partial'a model="Model" verilmesi, controller'dan alınmış paketin aynısını alt view'a taşır. Yeni controller isteği veya SQL sorgusu başlatmaz. Partial yalnızca sayfanın tekrar kullanılabilir HTML parçasıdır.

Layout=null nedeniyle bu view html/head/body kabuğunu kendisi kurar. Admin ekranında ise ortak _AdminLayout vardır. Html.Raw HTML string'ini markup olarak gösterir; @entry.Title normal metin kodlamasıyla gösterilir. Raw bir temizleyici değildir; gövdenin güvenli modeli ayrı değerlendirilir.

Sabit sayfanın BodyHtml içindeki h1 metni ile Title metadata'sı ayrı olabilir. Blog detayının ana h1'i ise @entry.Title ile doğrudan modelden gelir. Aynı isimli "başlık" alanları her görünümde aynı yoldan çizilmiyor.

_BlogListing: kart, filtre ve sayfalama

Hangi görünür öge modelin hangi alanından geliyor?

View ifadesi	Veri kaynağı	Ekrandaki karşılığı
Model.Posts	Controller'ın filtrelenmiş, sayfalanmış listesi	foreach ile kartlar
post.Title / post.Url	Listedeki ContentPage	Başlık ve detay bağlantısı
post.CoverImageUrl ?? varsayılan	Yazı kaydı / null durumunda fallback	Kart görseli
Model.Categories	Tüm yayınlanmış yazılardan tekil kategori adları	Filtre bağlantıları
Model.Category == category	Seçili filtre	selected CSS sınıfı
Model.TotalPosts	Filtreye uyan toplam kayıt	Yazı sayısı
PageNumber / TotalPages	Controller hesaplaması	Önceki/sonraki bağlantıları

```
@foreach (var post in Model.Posts)
{
    <h3><a href="@post.Url">@post.Title</a></h3>
    <p>@post.Description</p>
}
@if (Model.Posts.Count == 0)
{
    <p>Henüz yayınlanmış yazı yok.</p>
}
```

Sadeleştirilmiş örnek: foreach veritabanını tekrar taramaz; hazır listeyi dolaşır. Kart sayısı kadar SQL sorgusu yapılmaz. Model.ListingUrl(number, category), sayfalama URL'sini üretir; kategori değerini Uri.EscapeDataString ile URL'ye uygun kodlar. Bir sonraki sayfaya tıklamak ise yeni HTTP isteği başlatır.

Ana sayfada “Tüm yazıları gör” linki /blog’a gider. Blog listesinde sayfalama, kategori parametresini korur; kategori seçmek sayfa numarasını 1’e döndürür. Aksi halde önceki kategorideki yüksek sayfa numarası yeni filtre için anlamsız olabilir.

Başlık linkinin rengi CSS işidir. Hangi başlığın ve URL'nin geldiği model/controller işidir. Renk değiştirmek için migration gerekmez; kartta yeni kalıcı veri göstermek için veri yolunun tamamını düşünmek gerekir.

Bir alanın admin'den ziyaretçiye yolculuğu

View değişikliğinin sınırlarını somut örnekle görelim.

1. Admin/Edit.cshtml içindeki input asp-for="Author" kullanıcıya yazar adını düzenletir.
2. POST model binding, Author form değerini ContentPage input.Author'a bağlar.
3. AdminController.Save, doğrulamadan sonra input.Author değerini gerçek page.Author'a kopyalar.
4. SaveChangesAsync yeni değeri Pages tablosuna yazar.
5. Ziyaretçi /blog istediğinde SiteController güncel kayıtları Posts listesine alır.
6. _BlogListing.cshtml @post.Author ile yeni metni gösterir.

Bu akış için SiteController'a yeni sorgu yazmak gerekmez; Author entity üzerinde zaten vardır ve mevcut sorgu entity'yi getirir. Öte yandan alan hiç yoksa yalnızca Razor'a @post.ReadingMinutes yazmak yetmez; model ve kalıcılık tasarımı gerekir.

İstenen değişiklik	Gereken çalışma
Label "Yazar" yerine "Yazıyı hazırlayan" olsun	Label metni; asp-for aynı kalır.
Yazar adı italik gösterilsin	Ziyaretçi view'ı/CSS.
Yeni bir okuma süresi verisi saklansın	Entity alanı, migration, admin input, Save ataması, view gösterimi, test.
Bir alan ekranda farklı adla taşınsın	Ayrı view model eşlemesi; DB değişmiyorsa migration yok.

Admin controller rehberi bu kayıt zincirini ayrıntılı ele alır. SiteController'ın sorumluluğu kaydetmek değil, verinin sonraki istekte doğru filtre ve ekran modeliyle tekrar okunmasıdır.

Öğrenme kontrolü: "Formda görünen değer" ile "DB'de kayıtlı değer" her zaman aynı olmayabilir. Hatalı POST, kullanıcının girdilerini view'da koruyabilir ama kaydetmez. Başarılı kayıt sonrası ziyaretçi sorgusu kalıcı değeri getirir.

Kodun tamamı · ilk bölüm

Açıklamalardan sonra gerçek dosyayı bir bütün olarak okuyun.

```

01 using Microsoft.AspNetCore.Mvc;
02 using Microsoft.EntityFrameworkCore;
03 using Voku.Web.Data;
04 using Voku.Web.Models;
05 namespace Voku.Web.Controllers;
06 public class SiteController(SiteDbContext db) : Controller
07 {
08     private const int PageSize = 6;
09     [HttpGet("/")]
10     public Task<IActionResult> Home() => Render("home", false, null, 1);
11     [HttpGet("/{slug}")]
12     public Task<IActionResult> Page(string slug, [FromQuery] string? category, [FromQuery(Name =
"page")] int number = 1)
13         => Render(slug, false, category, number);
14     [HttpGet("/blog/{slug}")]
15     public Task<IActionResult> Detail(string slug) => Render(slug, true, null, 1);
16     [Route("/server-error")]
17     public IActionResult ServerError() => Problem("İşlem tamamlanamadı. Lütfen tekrar deneyin.");
18
19     private async Task<IActionResult> Render(string slug, bool detail, string? category, int number)
20     {
21         var page = await db.Pages.AsNoTracking().SingleOrDefaultAsync(p => p.Slug == slug &&
p.IsDetail == detail && p.Published);
22         if (page == null)
23         {
24             Response.StatusCode = 404;
25             page = await db.Pages.AsNoTracking().SingleOrDefaultAsync(p => p.Slug == "error" &&
!p.IsDetail && p.Published);
26             if (page == null) return NotFound();
27         }

```

İlk bölüm: bağımlılıklar, route'lar, ortak Render metodu, kayıt seçimi ve bulunamama davranışı. Buradaki satır numaraları hazırlama anındaki gerçek dosyayla eşleşir.

Kodun tamamı · ikinci bölüm

Listeleme, detay ve genel ekran dönüşleri.

```

28         if (!page.IsDetail && page.Slug is "home" or "blog")
29         {
30             var query = db.Pages.AsNoTracking().Where(p => p.IsDetail && p.Published);
31             var categories = await query.Where(p => p.Category != null && p.Category != "")
32                 .Select(p => p.Category!).Distinct().OrderBy(c => c).ToListAsync();
33             category = page.Slug == "blog" ? category?.Trim() : null;
34             if (!string.IsNullOrEmpty(category)) query = query.Where(p => p.Category == category);
35             var count = await query.CountAsync();
36             var totalPages = (int)Math.Ceiling(count / (double)PageSize);
37             number = Math.Clamp(number, 1, Math.Max(1, totalPages));
38             var posts = await query.OrderByDescending(p => p.PublishedAtUtc).ThenByDescending(p => p.Id)
39                 .Skip((number - 1) * PageSize).Take(PageSize).ToListAsync();
40             return View("Page", new SitePageViewModel { Page = page, Posts = posts, Categories = categories,
41                 Category = category, PageNumber = number, TotalPages = totalPages, TotalPosts = count });
42         }
43         if (page.IsDetail)
44         {
45             var home = await db.Pages.AsNoTracking().SingleOrDefaultAsync(p => p.Slug == "home" &&
!p.IsDetail);
46             return View("Page", new SitePageViewModel { Page = page,
47                 HeaderHtml = BlogMarkup.Chrome(home?.BodyHtml ?? page.BodyHtml, "header"),
48                 FooterHtml = BlogMarkup.Chrome(home?.BodyHtml ?? page.BodyHtml, "footer") });
49         }
50         return View("Page", new SitePageViewModel { Page = page });
51     }
52 }

```

Kod referansı: [SiteController.cs · 4c8cf8c](#). Yardımcı model: [SitePageViewModel.cs](#). Bu ders, uygulama kodunda değişiklik yapmaz.